

# Map and Set - Simple Hinglish Summary

Source: <https://javascript.info/map-set>

## Main baat

Ab tak complex data structures me humne dekha:

- **Objects:** keyed collections ke liye
- **Arrays:** ordered collections ke liye

Lekin real life ke liye ye kaafi nahi. Isliye **Map** aur **Set** bhi hain.

## 1. Map

**Map** keyed data items ka collection hai, bilkul **Object** jaisa. **Main fark:** **Map** me **kisi bhi type ki key** ho sakti hai.

### Methods aur properties:

- **new Map()**: map banata hai
- **map.set(key, value)**: key ke saath value store karta hai
- **map.get(key)**: key se value return karta hai, key na ho to **undefined**
- **map.has(key)**: key ho to **true**, warna **false**
- **map.delete(key)**: key wala element hata deta hai
- **map.clear()**: map me se sab kuch hata deta hai
- **map.size**: abhi ke elements ki ginti

javascript

```
let map = new Map();

map.set('1', 'str1'); // string key
map.set(1, 'num1');   // numeric key
map.set(true, 'bool1'); // boolean key

// regular Object keys ko string me badal deta hai
// Map type banaye rakhta hai, isliye ye dono alag hain:
alert( map.get(1) ); // 'num1'
alert( map.get('1') ); // 'str1'

alert( map.size ); // 3
```

Objects ke ulta, **keys strings me convert nahi hoti**. Koi bhi type ki key ho sakti hai.

`map[key]` `Map` use karne ka sahi tarika nahi hai. Ye kaam to karta hai (`map[key] = 2`), lekin tab `map` ko plain JS object ki tarah treat kar rahe ho, jisse uski saari limitations aa jati hain (sirf string/symbol keys, etc.). Isliye `set`, `get` jaise `map` methods use karo.

### Map me objects keys ho sakte hain

```
javascript

let john = { name: "John" };

// har user ke visits count store karte hain
let visitsCountMap = new Map();

// john map ki key hai
visitsCountMap.set(john, 123);

alert( visitsCountMap.get(john) ); // 123
```

Objects ko keys banana `Map` ki sabse notable aur important features me se ek hai.

`Object` me ye nahi ho sakta. Object me string key theek hai, lekin dusra object key nahi bana sakte. Try karo:

```
javascript

let john = { name: "John" };
let ben = { name: "Ben" };

let visitsCountObj = {}; // object use karne ki koshish

visitsCountObj[ben] = 234; // ben object ko key banaya
visitsCountObj[john] = 123; // john object ko key banaya, ben wala replace ho j

// Ye likha gaya!
alert( visitsCountObj["[object Object]"] ); // 123
```

`visitsCountObj` ek object hai, isliye wo saari `Object` keys (jaise `john` aur `ben`) ko ek hi string `"[object Object]"` me badal deta hai. Bilkul wo nahi jo hum chahte the.

`Map` keys ko kaise compare karta hai: Keys barabar hain ya nahi, ye check karne ke liye `Map` `SameValueZero` algorithm use karta hai. Ye lagbhag strict equality `===` jaisa hai, farak sirf ye ki `NaN` ko `NaN` ke barabar maana jata hai. Isliye `NaN` bhi key ban sakta hai. Is algorithm ko change ya customize nahi kar sakte.

**Chaining:** har `map.set` call map ko hi return karti hai, isliye calls ko "chain" kar sakte hain:

javascript

```
map.set('1', 'str1')  
  .set(1, 'num1')  
  .set(true, 'bool1');
```

## 2. Map par iteration

`map` par loop chalane ke **3 methods** hain:

- `map.keys()`: keys ka iterable
- `map.values()`: values ka iterable
- `map.entries()`: entries `[key, value]` ka iterable. `for..of` me ye **by default** use hota hai.

javascript

```
let recipeMap = new Map([  
  ['cucumber', 500],  
  ['tomatoes', 350],  
  ['onion', 50]  
]);  
  
// keys par iterate (sabziyan)  
for (let vegetable of recipeMap.keys()) {  
  alert(vegetable); // cucumber, tomatoes, onion  
}  
  
// values par iterate (matra)  
for (let amount of recipeMap.values()) {  
  alert(amount); // 500, 350, 50  
}  
  
// [key, value] entries par iterate  
for (let entry of recipeMap) { // recipeMap.entries() ke barabar  
  alert(entry); // cucumber,500 (aur aage)  
}
```

**Insertion order use hota hai:** iteration usi order me hoti hai jisme values daali gayi thi. Regular `Object` ke ulta, `Map` ye order banaye rakhta hai.

`Map` me `Array` jaisa built-in `forEach` bhi hai:

javascript

```
// har (key, value) pair ke liye function chalata hai
recipeMap.forEach( (value, key, map) => {
  alert(`${key}: ${value}`); // cucumber: 500 etc
});
```

### 3. `Object.entries`: Object se Map

`Map` banate waqt **key/value pairs ka array** (ya koi aur iterable) initialization ke liye de sakte hain:

```
javascript

// [key, value] pairs ka array
let map = new Map([
  ['1', 'str1'],
  [1, 'num1'],
  [true, 'bool1']
]);

alert( map.get('1') ); // str1
```

Agar plain object se `Map` banana ho to built-in `Object.entries(obj)` use karo, jo object ke liye bilkul isi format me key/value pairs ka array return karta hai:

```
javascript

let obj = {
  name: "John",
  age: 30
};

let map = new Map(Object.entries(obj));

alert( map.get('name') ); // John
```

`Object.entries` yahan `[ ["name", "John"], ["age", 30] ]` return karta hai. `Map` ko yahi chahiye.

### 4. `Object.fromEntries`: Map se Object

`Object.entries(obj)` se object se `Map` banaya. `Object.fromEntries` ulta karta hai: `[key, value]` pairs ke array se **object** banata hai:

```
javascript
```

```
let prices = Object.fromEntries([
  ['banana', 1],
  ['orange', 2],
  ['meat', 4]
]);

// ab prices = { banana: 1, orange: 2, meat: 4 }

alert(prices.orange); // 2
```

`Map` se plain object pane ke liye bhi `Object.fromEntries` use kar sakte hain. Jaise data `Map` me store hai, lekin kisi 3rd-party code ko plain object chahiye:

javascript

```
let map = new Map();
map.set('banana', 1);
map.set('orange', 2);
map.set('meat', 4);

let obj = Object.fromEntries(map.entries()); // plain object banao (*)

// ho gaya!
// obj = { banana: 1, orange: 2, meat: 4 }

alert(obj.orange); // 2
```

`map.entries()` key/value pairs ka iterable deta hai, bilkul `Object.fromEntries` ke format me.

Line `(*)` chhoti bhi kar sakte hain:

javascript

```
let obj = Object.fromEntries(map); // .entries() hata do
```

Wahi hai, kyunki `Object.fromEntries` ko argument me iterable chahiye (array hona zaruri nahi). Aur `map` ki standard iteration wahi key/value pairs deti hai jo `map.entries()`.

## 5. Set

`Set` ek special collection hai: "**values ka set**" (bina keys ke), jahan **har value sirf ek baar** aa sakti hai.

**Main methods:**

- `new Set([iterable])`: set banata hai, aur iterable (aam taur par array) diya ho to usse values copy karta hai
- `set.add(value)`: value jodta hai, set ko hi return karta hai
- `set.delete(value)`: value hatata hai, value thi to `true`, warna `false`
- `set.has(value)`: value ho to `true`
- `set.clear()`: sab kuch hata deta hai
- `set.size`: elements ki ginti

**Main khoobi:** same value ke saath `set.add(value)` ke baar baar calls kuch nahi karte. Isi wajah se har value `Set` me sirf ek baar hoti hai.

Jaise visitors aa rahe hain aur hume sabko yaad rakhna hai, lekin baar baar aane par duplicates nahi chahiye. Ek visitor sirf ek baar "ginna" hai. `Set` bilkul sahi hai:

javascript

```
let set = new Set();

let john = { name: "John" };
let pete = { name: "Pete" };
let mary = { name: "Mary" };

// visits, kuch users kai baar aate hain
set.add(john);
set.add(pete);
set.add(mary);
set.add(john);
set.add(mary);

// set sirf unique values rakhta hai
alert( set.size ); // 3

for (let user of set) {
  alert(user.name); // John (phir Pete aur Mary)
}
```

`Set` ka alternative users ka array hota, aur har insertion par `arr.find` se duplicates check karna. Lekin **performance bahut kharab hoti**, kyunki ye method har baar poora array chalata hai. `Set` andar se uniqueness checks ke liye bahut behtar optimized hai.

## 6. Set par iteration

`for...of` ya `forEach` se loop chala sakte hain:

javascript

```
let set = new Set(["oranges", "apples", "bananas"]);

for (let value of set) alert(value);

// forEach se wahi:
set.forEach((value, valueAgain, set) => {
  alert(value);
});
```

**Mazedar baat:** `forEach` ke callback me **3 arguments** hote hain: `value`, phir wahi `value` `valueAgain`, phir target object. Yaani wahi value do baar aati hai.

Ye `Map` ke saath **compatibility** ke liye hai, jahan `forEach` ke callback me teen arguments hote hain. Thoda ajeeb lagta hai, lekin isse kabhi kabhi `Map` ko `Set` se (aur ulta) aasani se badal sakte hain.

`Map` ke jaise iterator methods `Set` me bhi hain:

- `set.keys()`: values ka iterable
- `set.values()`: `set.keys()` jaisa, `Map` ke saath compatibility ke liye
- `set.entries()`: entries `[value, value]` ka iterable, `Map` ke saath compatibility ke liye

## Summary

### `Map` - keyed values ka collection

- `new Map([iterable])`: map banata hai, optional `[key, value]` pairs ke iterable (jaise array) ke saath
- `map.set(key, value)`: key ke saath value store karta hai, map ko return karta hai
- `map.get(key)`: key se value, key na ho to `undefined`
- `map.has(key)`: key ho to `true`
- `map.delete(key)`: key wala element hatata hai, key thi to `true`
- `map.clear()`: sab hata deta hai
- `map.size`: elements ki ginti

**Regular `Object` se farak:**

- **Koi bhi key**, objects bhi keys ho sakte hain
- Extra convenient methods, `size` property

### `Set` - unique values ka collection

- `new Set([iterable])`: set banata hai, optional values ke iterable (jaise array) ke saath
- `set.add(value)`: value jodta hai (pehle se ho to kuch nahi), set ko return karta hai

- `set.delete(value)`: value hatata hai, value thi to `true`
- `set.has(value)`: value ho to `true`
- `set.clear()`: sab hata deta hai
- `set.size`: elements ki ginti

`Map` aur `Set` par iteration hamesha **insertion order** me hoti hai. Isliye inhe "unordered" nahi keh sakte, lekin elements ko reorder nahi kar sakte aur number se seedha element nahi le sakte.

## Practice Tasks (Answers)

### 1. Array ke unique members (`Set` se):

javascript

```
function unique(arr) {
  return Array.from(new Set(arr));
}

let values = ["Hare", "Krishna", "Hare", "Krishna",
  "Krishna", "Krishna", "Hare", "Hare", ":-0"];

alert( unique(values) ); // Hare, Krishna, :-0
```

### 2. Anagrams filter karo (`aclean`): Anagrams: wo words jinme same letters same number me hote hain, bas order alag. Jaise `nap - pan`, `ear - are - era`.

**Idea:** har word ko letters me todo, sort karo, wapas jodo. Sorted form me saare anagrams same ban jate hain (`nap, pan`) → `anp`). Is sorted form ko **map ki key** banao taaki har key par sirf ek word rahe:

javascript

```
function aclean(arr) {
  let map = new Map();

  for (let word of arr) {
    // word ko letters me todo, sort karo, wapas jodo
    let sorted = word.toLowerCase().split('').sort().join(''); // (*)
    map.set(sorted, word);
  }

  return Array.from(map.values());
}

let arr = ["nap", "teachers", "cheaters", "PAN", "ear", "era", "hectares"];

alert( aclean(arr) ); // "nap,teachers,ear" ya "PAN,cheaters,era"
```

Line `(*)` ko samjho:

javascript

```
let sorted = word // PAN
  .toLowerCase() // pan
  .split('')      // ['p','a','n']
  .sort()         // ['a','n','p']
  .join('');      // anp
```

Dobara wahi sorted form mile to purani value overwrite ho jati hai, isliye har letter-form ka maximum ek word bachta hai. End me `Array.from(map.values())` keys nahi, sirf values ka array deta hai.

(Yahan keys strings hain, isliye `Map` ki jagah plain object bhi chal sakta tha.)

**3. Iterable keys (`keys.push`) kaam kyu nahi karta?):**

javascript

```
let map = new Map();

map.set("name", "John");

let keys = map.keys();

// Error: keys.push is not a function
keys.push("more");
```

**Kyunki** `map.keys()` **iterable** return karta hai, **array nahi**. **Fix:** `Array.from` se array banao:

javascript

```
let keys = Array.from(map.keys());
```

```
keys.push("more");
```

```
alert(keys); // name, more
```

## Quick Summary

| Baat    | Object                                          | Map                                                                        |
|---------|-------------------------------------------------|----------------------------------------------------------------------------|
| Keys    | Sirf string/symbol                              | Koi bhi type (objects bhi)                                                 |
| Order   | Special (integer keys sorted)                   | Insertion order                                                            |
| Size    | Manual ( <code>Object.keys(obj).length</code> ) | <code>map.size</code>                                                      |
| Iterate | <code>Object.keys/values/entries</code>         | <code>map.keys()</code> , <code>.values()</code> , <code>.entries()</code> |

| Kaam                    | Kaise                                                         |
|-------------------------|---------------------------------------------------------------|
| Map banana              | <code>new Map()</code> ya <code>new Map([[k, v], ...])</code> |
| Value rakhna / lena     | <code>map.set(k, v)</code> / <code>map.get(k)</code>          |
| Key hai ya nahi         | <code>map.has(k)</code>                                       |
| Hatana                  | <code>map.delete(k)</code> , <code>map.clear()</code>         |
| Object → Map            | <code>new Map(Object.entries(obj))</code>                     |
| Map → Object            | <code>Object.fromEntries(map)</code>                          |
| Set banana              | <code>new Set([1, 2, 2, 3])</code>                            |
| Unique values           | <code>Array.from(new Set(arr))</code>                         |
| Set me add / check      | <code>set.add(v)</code> / <code>set.has(v)</code>             |
| Map ki keys ko array me | <code>Array.from(map.keys())</code>                           |

**Yaad rakho:**

- `map[key]` mat use karo, hamesha `map.set/get`.
- `Set` me duplicates apne aap hat jate hain.
- `map.keys()` iterable hai, array nahi.